

Acelerador em FPGA para o Supervisor de Segurança

LiDAR de uma Cadeira de Rodas Semi-Assistida:

Ponto de Controle 2 — Implementação RTL e Verificação por Equivalência

Vítor Gonçalves de Andrade Silva
Engenharia Eletrônica — FCTE/UnB
Email: 261025182@aluno.unb.br

Thiago Henrique Oliveira Souza
Engenharia Eletrônica — FCTE/UnB
Email: 180131672@aluno.unb.br

Resumo—Este segundo ponto de controle implementa em VHDL a arquitetura proposta em PC1 — um CORDIC pipelinado de 16 estágios alimentando 19 pistas paralelas de coeficiente constante — e a verifica por equivalência bit a bit contra um modelo de referência, sobre três varreduras *reais* capturadas na cadeira. As três passam com zero divergência. A verificação expôs o risco antecipado em PC1: um defeito de faixa no formato Q3.13. O ângulo do feixe atinge $-5,85$ rad e não cabe em $\pm 4,0$ rad, saturando e projetando o ponto no lugar errado (erro de 402 mm); envolvendo o ângulo em $[-\pi, \pi]$ antes da quantização, o erro cai a 0,99 mm médio — cerca de um LSB. A latência, medida na simulação GHDL, é de 751 ciclos (7,51 μ s a 100 MHz); contra o *baseline* medido na própria Raspberry Pi 5 (3,54 ms escalar, 0,488 ms vetorizado), $472\times$ e $65\times$. A síntese — recursos, *timing*, potência e integração em SoC — ainda não foi executada e é o objeto do ponto de controle final.

I. RECAPITULAÇÃO

Em PC1 identificamos, por perfilamento, o teste de corredor do supervisor de segurança como o bloco a acelerar: é o nó mais pesado (17,7% da CPU da Pi 5) e o único cuja latência tem consequência física, pois só corta o avanço e a cadeira ainda percorre distância por inércia [1]. O algoritmo projeta os 720 feixes no referencial *base_link* e testa, para 19 direções candidatas δ_k , se cada ponto cai no corredor físico da cadeira (largura $W = 0,61$ m, frente $X_f = 0,667$ m) — 9918 avaliações por varredura, custo $O(N \cdot K)$. Propusemos migrar esse custo para hardware e manter a função custo $O(K)$ no ARM, onde vivem os pesos ajustados em campo. Este documento implementa e verifica; a síntese fica para PC3.

II. IMPLEMENTAÇÃO RTL

O RTL está organizado em quatro módulos VHDL (Tab. I), do CORDIC ao envelope AXI, todos parametrizáveis pelos formatos fixados em PC1.

a) **CORDIC**: Implementado em modo rotação [3], pipelinado em 16 estágios, usando apenas somadores e deslocamentos cabeados — sem nenhum multiplicador. Como o laço só converge para $|z| \leq 1,7434$ rad, um estágio de redução de quadrante traz θ para $[-\pi/2, \pi/2]$ e inverte o sinal na saída. A opção por CORDIC, e não por LUT de seno, se justifica porque *angle_min*, o incremento e o *yaw* são parâmetros de calibração reescritos em tempo de execução via AXI4-Lite

Tabela I
MÓDULOS RTL IMPLEMENTADOS

Arquivo	Função
<code>cordic_pkg.vhd</code>	constantes (geradas pelo modelo de ouro)
<code>cordic.vhd</code>	CORDIC rotação, 16 estágios, 0 DSP
<code>lane.vhd</code>	uma pista: rotação de coef. constante + teste
<code>corridor_core.vhd</code>	projeção + 19 pistas + redução de mínimo
<code>lidar_accel_axi.vhd</code>	envelope AXI4-Stream (feixes) + AXI4-Lite (ctrl/folgas)

— uma ROM indexada pelo índice do feixe exigiria fixá-los em síntese.

b) **Pistas**: Em cada pista, $\cos \delta_k$ e $\sin \delta_k$ são *generics* constantes de síntese; a rotação vira quatro multiplicações de coeficiente fixo. O *core* instancia as 19 pistas em paralelo, de modo que a vazão alvo de **1 feixe/ciclo** se mantém: cada feixe projetado é avaliado simultaneamente pelos 19 candidatos.

c) **Constantes geradas, não transcritas**: As tabelas de *atan*, o ganho do CORDIC e os coeficientes das pistas em `cordic_pkg.vhd` são **gerados** por `golden/gen_pkg.py` a partir do mesmo modelo que serve de referência. Assim, RTL e referência casam *por construção*: um erro de digitação numa tabela de constantes seria, de outro modo, indetectável por inspeção.

III. MODELO DE OURO E METODOLOGIA DE VERIFICAÇÃO

Conforme planejado em PC1, a verificação é por **equivalência bit a bit**, e não por inspeção de formas de onda:

- 1) um **modelo em ponto fixo** (`golden/model.py`) emula o RTL exatamente — mesmo CORDIC, mesmos deslocamentos, mesmos truncamentos;
- 2) um **modelo em ponto flutuante** serve de referência física, para medir o erro de quantização;
- 3) o modelo gera o **estímulo** e o **esperado**; o testbench VHDL (`tb/tb_corridor_core.vhd`) lê os arquivos, injeta os 720 feixes e compara as 19 folgas;
- 4) o estímulo não é sintético: são três varreduras **reais** da cadeira, capturadas com um balde posicionado 21 cm

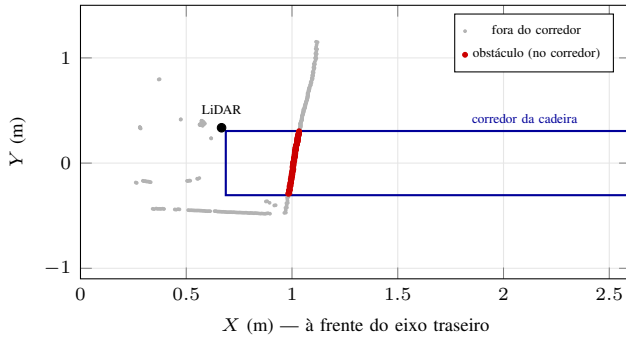


Figura 1. Varredura real projetada no `base_link` (estímulo do testbench). O LiDAR está 0,336 m à esquerda do eixo. Os pontos em vermelho caem no corredor que a cadeira ocupará — entre eles o balde, que o cone antigo não enxergava.

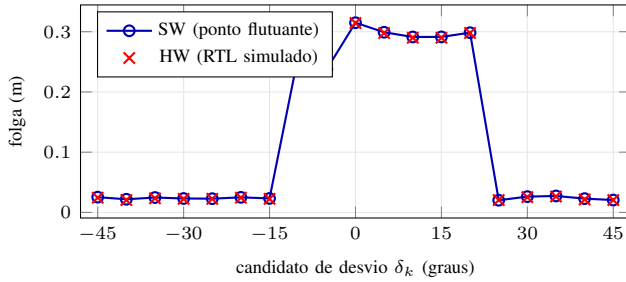


Figura 2. Folga por candidato: software *versus* saída real do RTL, para a varredura de teste. As curvas se sobrepõem em toda a faixa — o desvio é invisível nesta escala, precisamente o resultado desejado.

à direita do eixo — exatamente o antigo ponto cego (Fig. 1).

IV. RESULTADOS DE SIMULAÇÃO

A. Equivalência

As três varreduras reais passam pelo testbench com **zero divergência**: para cada uma, as 19 folgas produzidas pelo RTL simulado coincidem com o esperado gerado pelo modelo de ponto fixo. A Fig. 2 confronta, por candidato, a folga do software (ponto flutuante, a referência física) com a saída efetiva da simulação VHDL; as curvas se sobrepõem.

B. Precisão: um defeito de formato

A análise de precisão confirmou o risco levantado em PC1 e produziu o achado mais instrutivo desta fase. O primeiro modelo em ponto fixo divergia da referência em até 402 mm — inaceitável num sistema de segurança, e grande demais para ser arredondamento.

A causa não era ruído de quantização, e sim **estouro de faixa**. O ângulo do feixe no referencial do sensor, $\theta_i = a_{\min} + i \cdot \Delta a + \theta_L$, atinge $-5,85$ rad; o formato Q3.13 comporta apenas $\pm 4,0$ rad. O valor **saturava**, e o feixe era projetado numa direção completamente errada. Envolvendo θ em $[-\pi, \pi)$ antes da quantização — operação barata, feita no host — o erro desaba (Tab. II).

Tabela II
ERRO DO PONTO FIXO CONTRA A REFERÊNCIA EM PONTO FLUTUANTE

	erro médio	erro máximo
θ saturado (defeito)	17,89 mm	402,15 mm
θ envolvido (corrigido)	0,99 mm	5,27 mm

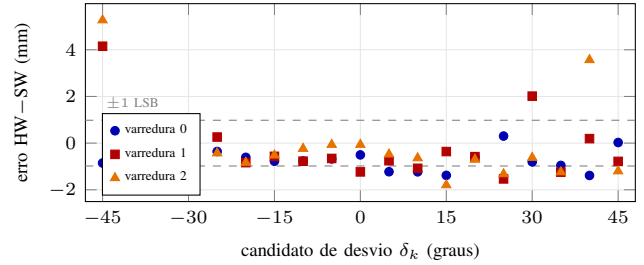


Figura 3. Resíduo HW-SW, com sinal, nas três varreduras (57 comparações). Linhas tracejadas: ± 1 LSB de Q6.10 (0,98 mm). O erro máximo, 5,27 mm, ocorre quando a quantização faz um ponto próximo da borda cruzar o limiar $|y| \leq W/2$ — efeito de um critério por limiar, não acúmulo aritmético.

Tabela III
TEMPO POR VARREDURA E GANHO (LATÊNCIA HW POR SIMULAÇÃO)

Implementação	Tempo	Speedup
Software escalar (Pi 5)	3,54 ms	—
Software vetorizado (NumPy)	0,488 ms	7,3×
HW 100 MHz (751 ciclos)	7,51 μs	472 × / 65×

O erro final é da ordem de **um LSB** de Q6.10 (0,98 mm) e representa 0,9 % da zona de parada de 600 mm: a quantização *não* é o fator limitante. A lição é que, em ponto fixo, a escolha da **faixa** costuma ser mais crítica que a da **resolução** — perdemos 402 milímetros por três bits de parte inteira, não por falta de bits fracionários. A Fig. 3 mostra o resíduo com sinal nas três varreduras.

C. Latência medida em simulação

A latência é o número de ciclos **contado na simulação** GHDL, não uma estimativa analítica: 751 ciclos por varredura, ou 7,51 μs a 100 MHz — próximo da previsão de PC1 ($\sim N$ mais a profundidade do *pipeline*). O *baseline* de software foi medido na própria Raspberry Pi 5 que embarca o sistema (Tab. III).

Vale a interpretação honesta, já possível nesta fase: o LiDAR entrega uma varredura a cada 100 ms, e mesmo o software vetorizado consome apenas 0,5 % desse orçamento. O acelerador não “salva” o sistema de perder prazos; seu valor é liberar CPU numa Pi que roda SLAM e EKF e, sobretudo, tornar a latência **determinística**. A quantificação definitiva desse ganho depende da síntese (potência, $F_{\max}$), ainda pendente.

V. ESTADO DA SÍNTESE — PENDENTE

A síntese ainda **não foi executada**. Mantemos, até PC3, apenas a estimativa analítica de PC1: 78 DSP48 (19×4 das pistas mais 2 da projeção) e 0 BRAM, por ser *streaming*. Esses

Tabela IV
O QUE FECHA EM PC3

Item	Estado após PC2
RTL (CORDIC, pista, <i>core</i> , AXI)	implementado
Verificação por equivalência	passa (3 varreduras reais)
Defeito de faixa Q3.13	encontrado e corrigido
Latência (simulação)	751 ciclos
Recursos, <i>timing</i> , potência	pendente (síntese em PC3)
<i>Block design</i> / gravação em placa	pendente (PC3)

números de recursos, o fechamento de *timing* (WNS, $F_{\max}$), a potência e a integração em *block design* com o ARM e o AXI-DMA constituem o escopo do ponto de controle final. Registramos a previsão aqui justamente para confrontá-la, em PC3, com o que a ferramenta efetivamente produzir.

VI. PRÓXIMOS PASSOS (PC3)

- 1) **Sintetizar** no xc7z020clg484-1 (ZedBoard) e extrair recursos, WNS/ $F_{\max}$ e potência; confrontar os DSP obtidos com a previsão de 78.
- 2) **Integrar em SoC**: empacotar o acelerador como IP e montá-lo com o ARM Cortex-A9 e o AXI-DMA num *block design*.
- 3) **Gravar em placa** e exercitar o driver do ARM (sw/lidar_accel.c) sobre o AXI-DMA, fechando o co-projeto.

VII. CONCLUSÃO PARCIAL

O RTL da arquitetura proposta em PC1 está escrito e **verificado por equivalência bit a bit** contra um modelo de referência, sobre dados reais da cadeira: as três varreduras passam. A verificação confirmou o risco que havíamos sinalizado — a faixa do formato de ângulo — e o converteu no resultado mais transferível desta fase: um ângulo de $-5,85$ rad saturando em Q3.13 produziu 402 mm de erro *silencioso*, que só a comparação com uma referência física revelou. A latência medida em simulação (751 ciclos) confirma a ordem de grandeza prevista. Resta a síntese — recursos, *timing*, potência e integração em SoC —, objeto do ponto de controle final.

REFERÊNCIAS

- [1] L. Fehr, W. E. Langbein e S. B. Skaar, “Adequacy of power wheelchair control interfaces for persons with severe disabilities: A clinical survey,” *J. Rehabil. Res. Dev.*, vol. 37, no. 3, pp. 353–360, 2000.
- [2] J. Borenstein e Y. Koren, “The Vector Field Histogram — Fast Obstacle Avoidance for Mobile Robots,” *IEEE Trans. Robot. Autom.*, vol. 7, no. 3, pp. 278–288, 1991.
- [3] J. E. Volder, “The CORDIC Trigonometric Computing Technique,” *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [4] R. Andraka, “A survey of CORDIC algorithms for FPGA based computers,” em *Proc. ACM/SIGDA Int. Symp. FPGAs*, 1998, pp. 191–200.
- [5] L. H. Crockett, R. A. Elliot, M. A. Enderwitz e R. W. Stewart, *The Zynq Book: Embedded Processing with the ARM Cortex-A9 on the Xilinx Zynq-7000*. Glasgow: Strathclyde Academic Media, 2014.
- [6] Open Robotics, “ROS 2 Jazzy Jalisco Documentation,” 2024. [Online]. Disponível: <https://docs.ros.org/en/jazzy/>